

PRÁCTICA DE LABORATORIO — SISTEMAS OPERATIVOS

Unidades V y VI: Gestión de E/S y Entornos de Virtualización

Modalidad	En pareja — cada integrante trabaja desde su propio equipo físico
-----------	---

Herramienta	VirtualBox 7.x + Ubuntu 24.04 LTS
-------------	-----------------------------------

Duración	2 semanas — entrega al inicio de la clase de la semana 3
----------	--

Integrante A	Nombre: Gabriel Antonio Pichardo Raban PC: username@ubuntu
--------------	---

Integrante B	Nombre: Mendelsson Alejandro Nuñez Montoya PC: mendel@mubuntu
--------------	--

Actividad Base:

Preparación de la Virtual Box (VM):

- SO: Ubuntu 24.04 LTS — RAM: 5gb — Disco: 40,50 GB dinámico
- CPU: 3 núcleos con Nested VT-x / AMD-V habilitado
- Red: Adaptador 1 en NAT + Adaptador 2 en Adaptador solo anfitrión Host-Only para acceso SSH desde el anfitrión

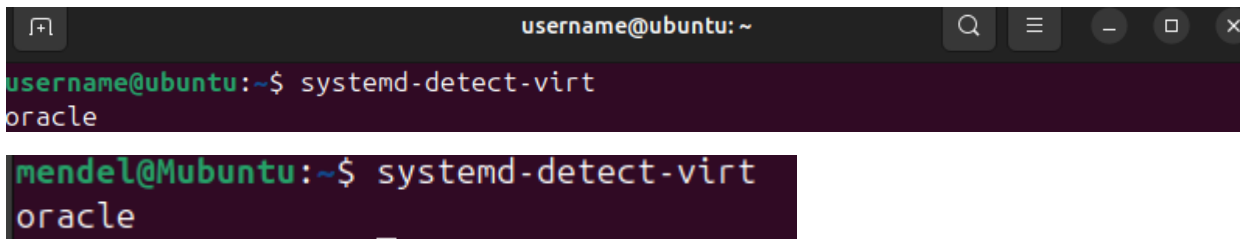
General Nombre: ubuntu Sistema operativo: Ubuntu (64-bit)	Previsualización ubuntu
Sistema Memoria base: 5052 MB Procesadores: 3 Orden de arranque: Disco duro, Óptica, Disquete Aceleración: Paginación anidada, Nested VT-x/AMD-V, Paravirtualización KVM	
Pantalla Memoria de vídeo: 16 MB Controlador gráfico: VMSVGA Servidor de escritorio remoto: Inhabilitado Grabación: Inhabilitado	
Almacenamiento Controlador: IDE Dispositivo IDE primario 0: [Unidad óptica] Vacío Controlador: SATA Puerto SATA 0: ubuntu.vdi (Normal, 40,50 GB)	
Audio Controlador de anfitrión: Predeterminado Controlador: ICH AC97	
Red Adaptador 1: Intel PRO/1000 MT Desktop (NAT) Adaptador 2: Intel PRO/1000 MT Desktop (Adaptador solo anfitrión, «VirtualBox Host-Only Ethernet Adapter»)	



Clasificación del hypervisor (Tema 6.1)

1. Dentro de la VM: `systemd-detect-virt` y `lscpu | grep -i hypervisor`

Systemd-detect-virt:



2. En el ANFITRIÓN: `lsmod | grep vbox` y `modinfo vboxdrv`

Como mi sistema operativo principal es Windows 11, los comandos `lsmod` y `modinfo` que solicita la guía no aplican de forma nativa. Para evidenciar que los módulos de VirtualBox están cargados en el kernel de mi equipo físico, ejecuté el comando `Get-WmiObject Win32_SystemDriver | Where-Object {$_.Name -like "*vbox*"}` desde PowerShell. En mi captura de pantalla se pueden observar varios controladores que son el equivalente funcional al `vboxdrv` de Linux:

```
PS C:\Users\Mendelsson> Get-WmiObject Win32_SystemDriver | Where-Object {$_.Name -like "*vbox*"}

DisplayName : VirtualBox NDIS 6.0 Miniport Service
Name        : VBoxNetAdp
State       : Running
Status      : OK
Started     : True

DisplayName : VirtualBox NDIS6 Bridged Networking Service
Name        : VBoxNetLwf
State       : Running
Status      : OK
Started     : True

DisplayName : VirtualBox Service
Name        : VBoxSup
State       : Running
Status      : OK
Started     : True

DisplayName : VirtualBox USB Monitor Service
Name        : VBoxUSBMon
State       : Running
Status      : OK
Started     : True
```

```
PS C:\Users\Carolina> Get-WmiObject Win32_SystemDriver | Where-Object {$_.Name -like "*vbox*"}

DisplayName : VirtualBox NDIS 6.0 Miniport Service
Name        : VBoxNetAdp
State       : Running
Status      : OK
Started     : True

DisplayName : VirtualBox NDIS6 Bridged Networking Service
Name        : VBoxNetLwf
State       : Running
Status      : OK
Started     : True

DisplayName : VirtualBox Service
Name        : VBoxSup
State       : Running
Status      : OK
Started     : True

DisplayName : VirtualBox USB Monitor Service
Name        : VBoxUSBMon
State       : Running
Status      : OK
Started     : True
```

¿Qué hace el módulo?: En un anfitrión Windows, los controladores listados (como VBoxSup, VBoxNetAdp, VBoxNetLwf y VBoxUSBMon) son equivalentes funcionales al módulo `vboxdrv` de Linux. VBoxSup es el servicio principal de VirtualBox que gestiona la ejecución de las máquinas virtuales, el acceso a la CPU y la memoria. VBoxNetAdp y VBoxNetLwf proporcionan las capacidades de red: el primero crea una interfaz de red solo-anfitrión (Host-Only), mientras que el segundo permite el modo puente al filtrar el tráfico en la pila de red de Windows. Finalmente, VBoxUSBMon es el monitor USB que redirige dispositivos USB desde el anfitrión hacia la VM. En conjunto, estos controladores permiten que VirtualBox ejecute sistemas operativos huéspedes aprovechando el hardware real, pero siempre bajo el control del kernel de Windows.

3. Conclusión justificada: ¿VirtualBox es Tipo 1 o Tipo 2? ¿Qué evidencia lo demuestra?

VirtualBox es un hypervisor de Tipo 2. La evidencia principal se observa en la captura del anfitrión: todos los componentes de VirtualBox aparecen como controladores o servicios que dependen del sistema operativo Windows. Esto demuestra que VirtualBox no se ejecuta directamente sobre el hardware, sino que se instala como una aplicación más dentro de Windows, y sus módulos se cargan a nivel del kernel del anfitrión. Además, al iniciar una VM, el

proceso de VirtualBox se ejecuta como un proceso de usuario más, y el hypervisor utiliza los controladores del host para acceder a la CPU y la memoria, lo cual es característico de los hypervisores de Tipo 2.

B. Exploración del Subsistema de E/S desde /proc y /sys

4. Cat /proc/interrupts:

```
mendel@Mubuntu:~$ cat /proc/interrupts
          CPU0           CPU1
 0:         126             0   IO-APIC  2-edge       timer
 1:           0          121   IO-APIC  1-edge       i8042
 8:           0             0   IO-APIC  8-edge       rtc0
 9:           0             0   IO-APIC  9-fasteoi    acpi
12:         158             0   IO-APIC 12-edge       i8042
14:        1714             0   IO-APIC 14-edge       ata_piix
15:           0             0   IO-APIC 15-edge       ata_piix
16:         894             0   IO-APIC 16-fasteoi    enp0s8
18:           0        3550   IO-APIC 18-fasteoi    vmwgfx
19:           0        1529   IO-APIC 19-fasteoi    ehci_hcd:usb2, enp0s3
20:        4995             0   IO-APIC 20-fasteoi    vboxguest
21:           0       12240   IO-APIC 21-fasteoi    ahci[0000:00:0d.0], snd_intel8x0
22:          27             0   IO-APIC 22-fasteoi    ohci_hcd:usb1
NMI:           0             0   Non-maskable interrupts
LOC:       136415       197183   Local timer interrupts
SPU:           0             0   Spurious interrupts
PMI:           0             0   Performance monitoring interrupts
IWI:           2             0   IRQ work interrupts
RTR:           0             0   APIC ICR read retries
RES:        10783       13272   Rescheduling interrupts
CAL:       38745       38139   Function call interrupts
TLB:        1217        1911   TLB shootdowns
TRM:           0             0   Thermal event interrupts
THR:           0             0   Threshold APIC interrupts
DFR:           0             0   Deferred Error APIC interrupts
MCE:           0             0   Machine check exceptions
MCP:           6             6   Machine check polls
ERR:           0
MIS:           0
PIN:           0             0   Posted-interrupt notification event
NPI:           0             0   Nested posted-interrupt event
PIW:           0             0   Posted-interrupt wakeup event
```

```

username@ubuntu:~$ cat /proc/interrupts

```

	CPU0	CPU1	CPU2			
0:	104	0	0	IO-APIC	2-edge	timer
1:	0	1099	0	IO-APIC	1-edge	i8042
8:	0	0	0	IO-APIC	8-edge	rtc0
9:	0	0	0	IO-APIC	9-fasteoi	acpi
12:	1346	0	0	IO-APIC	12-edge	i8042
14:	0	0	3022	IO-APIC	14-edge	ata_piix
15:	0	0	0	IO-APIC	15-edge	ata_piix
16:	2952	0	0	IO-APIC	16-fasteoi	enp0s8
18:	0	0	9	IO-APIC	18-fasteoi	vmwgfx
19:	0	0	621300	IO-APIC	19-fasteoi	ehci_hcd:usb2, enp0s3
20:	0	23297	0	IO-APIC	20-fasteoi	vboxguest
21:	157388	0	0	IO-APIC	21-fasteoi	ahci[0000:00:0d.0], snd_intel8x0
22:	0	26	0	IO-APIC	22-fasteoi	ohci_hcd:usb1
NMI:	0	0	0	Non-maskable interrupts		
LOC:	433844	1070211	548276	Local timer interrupts		
SPU:	0	0	0	Spurious interrupts		
PMI:	0	0	0	Performance monitoring interrupts		
IWI:	2	0	1	IRQ work interrupts		
RTR:	0	0	0	APIC ICR read retries		
RES:	18836	17259	19505	Rescheduling interrupts		
CAL:	359139	322183	293081	Function call interrupts		
TLB:	10922	11429	11406	TLB shootdowns		
TRM:	0	0	0	Thermal event interrupts		
THR:	0	0	0	Threshold APIC interrupts		
DFR:	0	0	0	Deferred Error APIC interrupts		
MCE:	0	0	0	Machine check exceptions		
MCP:	16	16	16	Machine check polls		
ERR:	0					
MIS:	0					
PIN:	0	0	0	Posted-interrupt notification event		

El comando Cat /proc/interrupts nos muestra detalladamente cómo los diferentes núcleos de la CPU gestionan y distribuyen las solicitudes de interrupción de hardware.

5. cat /proc/iomem:

```

mendel@Mubuntu:~$ cat /proc/iomem
00000000-00000000 : Reserved
00000000-00000000 : System RAM
00000000-00000000 : Reserved
00000000-00000000 : PCI Bus 0000:00
00000000-00000000 : Video ROM
00000000-00000000 : Adapter ROM
00000000-00000000 : Reserved
00000000-00000000 : System ROM
00000000-00000000 : System RAM
00000000-00000000 : Kernel code
00000000-00000000 : Kernel rodata
00000000-00000000 : Kernel data
00000000-00000000 : Kernel bss
00000000-00000000 : ACPI Tables
00000000-00000000 : PCI Bus 0000:00
00000000-00000000 : 0000:00:02.0
00000000-00000000 : vmwgfx probe
00000000-00000000 : 0000:00:02.0
00000000-00000000 : vmwgfx probe
00000000-00000000 : 0000:00:03.0
00000000-00000000 : e1000
00000000-00000000 : 0000:00:04.0
00000000-00000000 : vboxguest
00000000-00000000 : 0000:00:04.0
00000000-00000000 : 0000:00:06.0
00000000-00000000 : ohci_hcd
00000000-00000000 : 0000:00:08.0
00000000-00000000 : e1000
00000000-00000000 : 0000:00:0b.0
00000000-00000000 : ehci_hcd
00000000-00000000 : 0000:00:0d.0
00000000-00000000 : ahci
00000000-00000000 : Reserved
00000000-00000000 : IOAPIC 0
00000000-00000000 : Reserved
00000000-00000000 : Reserved
00000000-00000000 : System RAM

username@ubuntu:~$ cat /proc/iomem
00000000-00000000 : Reserved
00000000-00000000 : System RAM
00000000-00000000 : Reserved
00000000-00000000 : PCI Bus 0000:00
00000000-00000000 : Video ROM
00000000-00000000 : Adapter ROM
00000000-00000000 : Reserved
00000000-00000000 : System ROM
00000000-00000000 : System RAM
00000000-00000000 : Kernel code
00000000-00000000 : Kernel rodata
00000000-00000000 : Kernel data
00000000-00000000 : Kernel bss
00000000-00000000 : ACPI Tables
00000000-00000000 : PCI Bus 0000:00
00000000-00000000 : 0000:00:02.0
00000000-00000000 : vmwgfx probe
00000000-00000000 : 0000:00:02.0
00000000-00000000 : vmwgfx probe
00000000-00000000 : 0000:00:03.0
00000000-00000000 : e1000
00000000-00000000 : 0000:00:04.0
00000000-00000000 : vboxguest
00000000-00000000 : 0000:00:04.0
00000000-00000000 : 0000:00:06.0
00000000-00000000 : ohci_hcd
00000000-00000000 : 0000:00:08.0
00000000-00000000 : e1000
00000000-00000000 : 0000:00:0b.0
00000000-00000000 : ehci_hcd
00000000-00000000 : 0000:00:0d.0

```

El comando `cat /proc/iomem` muestra el mapa de asignación de la memoria física del sistema.

6. `cat /proc/ioports`:

```
mendel@Mubuntu:~$ cat /proc/ioports
0000-0000 : PCI Bus 0000:00
 0000-0000 : dma1
 0000-0000 : pic1
 0000-0000 : timer0
 0000-0000 : timer1
 0000-0000 : keyboard
 0000-0000 : keyboard
 0000-0000 : rtc_cmos
 0000-0000 : rtc0
 0000-0000 : dma page reg
 0000-0000 : pic2
 0000-0000 : dma2
 0000-0000 : fpu
 0000-0000 : 0000:00:01.1
 0000-0000 : ata_piix
 0000-0000 : 0000:00:01.1
 0000-0000 : ata_piix
 0000-0000 : 0000:00:01.1
 0000-0000 : ata_piix
 0000-0000 : vga+
 0000-0000 : 0000:00:01.1
 0000-0000 : ata_piix
0000-0000 : PCI conf1
0000-0000 : PCI Bus 0000:00
 0000-0000 : 0000:00:07.0
 0000-0000 : ACPI PM1a_EVT_BLK
 0000-0000 : ACPI PM1a_CNT_BLK
 0000-0000 : ACPI PM_TMR
 0000-0000 : ACPI GPE0_BLK
 0000-0000 : 0000:00:07.0
 0000-0000 : piix4_smbus
 0000-0000 : 0000:00:01.1
 0000-0000 : ata_piix
 0000-0000 : 0000:00:02.0
 0000-0000 : vmwgfx probe
 0000-0000 : 0000:00:03.0
 0000-0000 : e1000
 0000-0000 : 0000:00:04.0
 0000-0000 : 0000:00:05.0
 0000-0000 : Intel 82801AA-ICH
 0000-0000 : 0000:00:05.0
 0000-0000 : Intel 82801AA-ICH
 0000-0000 : 0000:00:08.0
 0000-0000 : e1000
 0000-0000 : 0000:00:0d.0
 0000-0000 : ahci
 0000-0000 : 0000:00:0d.0
 0000-0000 : ahci
 0000-0000 : 0000:00:0d.0
 0000-0000 : ahci
 0000-0000 : 0000:00:0d.0
 0000-0000 : ahci
 0000-0000 : 0000:00:0d.0
 0000-0000 : ahci
```

```
username@ubuntu:~$ cat /proc/ioports
0000-0000 : PCI Bus 0000:00
 0000-0000 : dma1
 0000-0000 : pic1
 0000-0000 : timer0
 0000-0000 : timer1
 0000-0000 : keyboard
 0000-0000 : keyboard
 0000-0000 : rtc_cmos
 0000-0000 : rtc0
 0000-0000 : dma page reg
 0000-0000 : pic2
 0000-0000 : dma2
 0000-0000 : fpu
 0000-0000 : 0000:00:01.1
 0000-0000 : ata_piix
 0000-0000 : 0000:00:01.1
 0000-0000 : ata_piix
 0000-0000 : 0000:00:01.1
 0000-0000 : ata_piix
 0000-0000 : vga+
 0000-0000 : 0000:00:01.1
 0000-0000 : ata_piix
0000-0000 : PCI conf1
0000-0000 : PCI Bus 0000:00
 0000-0000 : 0000:00:07.0
 0000-0000 : ACPI PM1a_EVT_BLK
 0000-0000 : ACPI PM1a_CNT_BLK
 0000-0000 : ACPI PM_TMR
 0000-0000 : ACPI GPE0_BLK
 0000-0000 : 0000:00:07.0
 0000-0000 : piix4_smbus
```

El comando `cat /proc/ioports` muestra el mapa de puertos de Entrada/Salida asignados a los dispositivos de hardware del sistema.

7. `cat /proc/devices`:

```

mendel@Mubuntu:~$ cat /proc/devices
Character devices:
 1 mem
 4 /dev/vc/0
 4 tty
 4 ttyS
 5 /dev/tty
 5 /dev/console
 5 /dev/ptmx
 5 ttyprintk
 6 lp
 7 vcs
10 misc
13 input
21 sg
29 fb
89 i2c
99 ppdev
108 ppp
116 alsa
128 ptm
136 pts
180 usb
189 usb_device
202 cpu/msr
204 ttyMAX
226 drm
240 hidraw
241 ttyDBC
242 bsg
243 watchdog
244 remoteproc
245 ptp
246 pps
247 rtc
248 dma_heap
249 dax
250 dimmctl
251 ndctl
252 tpm
253 pwm
254 gpiochip
261 accel

```

```

username@ubuntu:~$ cat /proc/devices
Character devices:
 1 mem
 4 /dev/vc/0
 4 tty
 4 ttyS
 5 /dev/tty
 5 /dev/console
 5 /dev/ptmx
 5 ttyprintk
 6 lp
 7 vcs
10 misc
13 input
21 sg
29 fb
89 i2c
99 ppdev
108 ppp
116 alsa
128 ptm
136 pts
180 usb
189 usb_device
202 cpu/msr
203 cpu/cpuid
204 ttyMAX
226 drm
240 hidraw
241 ttyDBC
242 bsg
243 watchdog
249 dax
250 dimmctl
251 ndctl
252 tpm
253 pwm
254 gpiochip
261 accel

Block devices:
 7 loop
 8 sd
 9 md
11 sr
65 sd
66 sd
67 sd
68 sd
69 sd
70 sd
71 sd
128 sd
129 sd
130 sd
131 sd
132 sd
133 sd
134 sd
135 sd
252 device-mapper
253 virtblk
254 mdp
259 blkext

```

El comando `cat /proc/devices` muestra la lista de controladores de dispositivos cargados en el kernel que están listos para comunicarse con el sistema.

8. `lspci -v | head -80`:


```
mende1@ubuntu:~$ lspci -v | head -80
00:00.0 Host bridge: Intel Corporation 440FX - 82441FX PMC [Natoma] (rev 02)
Flags: fast devsel

00:01.0 ISA bridge: Intel Corporation 82371SB PIIX3 ISA [Natoma/Triton II]
Flags: bus master, medium devsel, latency 0

00:01.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01) (prog-if 8a [ISA Compatibility mode controller, supports both channels switched to P
CI native mode, supports bus mastering])
Flags: bus master, fast devsel, latency 64
I/O ports at 01f0 [size=8]
I/O ports at 03f4 [size=8]
I/O ports at 0170 [size=8]
I/O ports at 0374 [size=8]
I/O ports at d000 [size=16]
Kernel driver in use: ata_piix
Kernel modules: pata_acpi

00:02.0 VGA compatible controller: VMware SVGA II Adapter (prog-if 00 [VGA controller])
Subsystem: VMware SVGA II Adapter
Flags: bus master, fast devsel, latency 64, IRQ 18
I/O ports at d010 [size=16]
Memory at e0000000 (32-bit, prefetchable) [size=128M]
Memory at f0000000 (32-bit, non-prefetchable) [size=2M]
Expansion ROM at 000c0000 [virtual] [disabled] [size=128K]
Kernel driver in use: vmwgfx
Kernel modules: vmwgfx

00:03.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
Flags: bus master, 66MHz, medium devsel, latency 64, IRQ 19
Memory at f0200000 (32-bit, non-prefetchable) [size=128K]
I/O ports at d020 [size=8]
Capabilities: <access denied>
Kernel driver in use: e1000
Kernel modules: e1000

00:04.0 System peripheral: InnoTek Systemberatung GmbH VirtualBox Guest Service
Flags: fast devsel, IRQ 20
I/O ports at d040 [size=32]
Memory at f0400000 (32-bit, non-prefetchable) [size=4M]
Memory at f0800000 (32-bit, prefetchable) [size=16K]
Kernel driver in use: vboxguest
Kernel modules: vboxguest
```

```
username@ubuntu:~$ lspci | head -80
00:00.0 Host bridge: Intel Corporation 440FX - 82441FX PMC [Natoma] (rev 02)
00:01.0 ISA bridge: Intel Corporation 82371SB PIIX3 ISA [Natoma/Triton II]
00:01.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01)
00:02.0 VGA compatible controller: VMware SVGA II Adapter
00:03.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
00:04.0 System peripheral: InnoTek Systemberatung GmbH VirtualBox Guest Service
00:05.0 Multimedia audio controller: Intel Corporation 82801AA AC'97 Audio Controller (rev 01)
00:06.0 USB controller: Apple Inc. KeyLargo/Intrepid USB
00:07.0 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
00:08.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
00:0b.0 USB controller: Intel Corporation 82801FB/FBM/FR/FW/FRW (ICH6 Family) USB2 EHCI Controller
00:0d.0 SATA controller: Intel Corporation 82801HM/HEM (ICH8M/ICH8M-E) SATA Controller [AHCI mode] (rev 02)
```

El comando `lspci | head -80` es la lista de forma detallada de todos los dispositivos y controladores conectados al bus PCI de la máquina

C. Kernel Module Individual

(Primer estudiante: Gabriel Pichardo)

1. Instalar cabeceras del kernel y herramientas de compilación:

```
sudo apt install linux-headers-$(uname -r) build-essential -y
```

```
username@ubuntu:~$ sudo apt install linux-headers-$(uname -r) build-essential -y
[sudo] password for username:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
linux-headers-6.17.0-23-generic is already the newest version (6.17.0-23.23~24.04.1).
build-essential is already the newest version (12.10ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 104 not upgraded.
```

2. Crear Makefile con: `obj-m := nombre_modulo.o`:

```
username@ubuntu: /practica_SO

username@ubuntu:~$ mkdir /practica_SO
mkdir: cannot create directory '/practica_SO': Permission denied
username@ubuntu:~$ sudo mkdir /practica_SO
[sudo] password for username:
username@ubuntu:~$ ls /practica_SO/
username@ubuntu:~$ ls
Desktop      Downloads      Music          Public         Templates
Documents    laboratorioSO  Pictures       snap           Videos
username@ubuntu:~$ cd /practica_SO/
username@ubuntu:/practica_SO$ touch makefilePichardo.c
touch: cannot touch 'makefilePichardo.c': Permission denied
username@ubuntu:/practica_SO$ sudo touch makefilePichardo.c
username@ubuntu:/practica_SO$ sudo touch Makefile
username@ubuntu:/practica_SO$ nano Makefile
username@ubuntu:/practica_SO$ sudo nano Makefile
username@ubuntu:/practica_SO$
```

Aquí procedemos a crear los archivos `io_timer_Pichardo.c` que es el archivo C y el archivo `Makefile`

```
username@ubuntu:/practica_SO$ sudo touch io_timer_Pichardo.c
```

```
GNU nano 7.2                                io_timer_Pichardo.c

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/ktime.h>
#include <linux/timekeeping.h>

#define AUTHOR_NAME "Gabriel Antonio Pichardo Raban"

MODULE_LICENSE("GPL");
MODULE_AUTHOR(AUTHOR_NAME);
MODULE_DESCRIPTION("Sistema de tiempo IO Timer");

// Este modulo nos sirve para inicializar y cargar nuestro modulo
int init_module(void){

    u64 time_ns;
    u64 uptime_ns;
    u64 diff_ns;

    printk(KERN_INFO "-----\n");

    //Obtenemos el timestamp en nano segundos
    time_ns = ktime_get_ns();

    //Ahora obtenemos el tiempo desde el arranque
    uptime_ns = ktime_get_boottime_ns();
}
```

```
username@ubuntu:/practica_SO
GNU nano 7.2 io_timer_Pichardo.c

//Ahora imprimimos ambos valores
printk(KERN_INFO "ktime_get() (Monotonico): %llu ns\n", time_ns);
printk(KERN_INFO "Uptime (Boottime): %llu ns\n", uptime_ns);

//Analizamos la consistencia de nuestros resultados
if(diff_ns > 0){
printk(KERN_INFO "Consistencia: Hay una diferencia de: %llu ns.\n", diff_ns);
}else{
printk(KERN_INFO "Consistencia: Los valores son iguales. \n");
}

return 0;
}

//Funcion de limpieza
void cleanup_module(void){

//Mandamos a mostrar en pantalla el mensaje de descarga con el nombre del autor
printk(KERN_INFO "Modulo io_timer Descargado. Autor %s\n", AUTHOR_NAME);
}
```

Nos metemos dentro del archivo C y colocamos las funciones `init_module` y `cleanup_module`, además de los módulos de License, Author y Description.

```
username@ubuntu:/practica_SO$ sudo touch Makefile
```

```
username@ubuntu:/practica_SO
GNU nano 7.2 Makefile
obj-m := io_timer_Pichardo.o
```

Ahora nos metemos en el archivo Makefile y lo “apuntamos” a nuestro archivo en C.

3. Ejecución:

```
username@ubuntu:/practica_SO$ sudo make -C /lib/modules/$(uname -r)/build M=$(pwd) modules
make: Entering directory '/usr/src/linux-headers-6.17.0-23-generic'
make[1]: Entering directory '/practica_SO'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] io_timer_Pichardo.o
MODPOST Module.symvers
CC [M] io_timer_Pichardo.mod.o
LD [M] io_timer_Pichardo.ko
BTF [M] io_timer_Pichardo.ko
Skipping BTF generation for io_timer_Pichardo.ko due to unavailability of vmlinux
make[1]: Leaving directory '/practica_SO'
make: Leaving directory '/usr/src/linux-headers-6.17.0-23-generic'
```

4. Carga y verificación de Mensajes printk:

```
username@ubuntu:/practica_S0$ sudo insmod io_timer_Pichardo.ko && sudo dmesg | tail -8
[12619.648272] e1000: enp0s3 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: RX
[13526.269341] audit: type=1400 audit(1780523192.203:198): apparmor="DENIED" operation="capable" class="cap" profile="/usr/sbin/cupsd" pid=23273 comm="cupsd" capability=12 capname="net_admin"
[13526.712218] audit: type=1400 audit(1780523192.648:199): apparmor="DENIED" operation="open" class="file" profile="snap.firmware-updater.firmware-notifier" name="/proc/sys/vm/max_map_count" pid=23174 comm="firmware-notifi" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[13598.190693] audit: type=1400 audit(1780523264.127:200): apparmor="STATUS" operation="profile_replace" info="same as current profile, skipping" profile="unconfined" name="/usr/sbin/sss" pid=24176 comm="apparmor_parser"
[15078.492564] -----
[15078.492574] ktime_get() (Monotonico): 15078669123098 ns
[15078.492577] Uptime (Boottime): 15078669123288 ns
[15078.492578] Consistencia: Hay una diferencia de: 190 ns.
```

5. Verificación de la lista de módulos:

```
username@ubuntu:/practica_S0$ sudo lsmod | grep io_timer_Pichardo
io_timer_Pichardo      12288      0
```

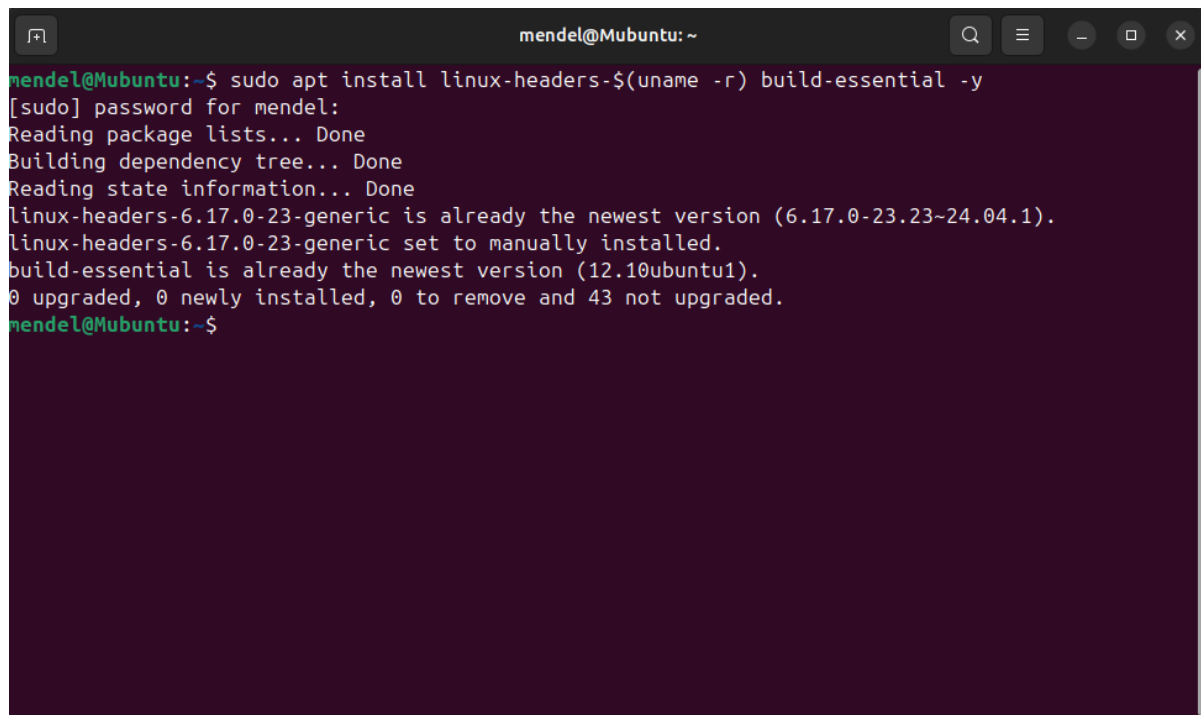
6. Descarga y verificación de mensaje cleanup:

```
username@ubuntu:/practica_S0$ sudo rmmod io_timer_Pichardo && sudo dmesg | tail -5
[15078.492564] -----
[15078.492574] ktime_get() (Monotonico): 15078669123098 ns
[15078.492577] Uptime (Boottime): 15078669123288 ns
[15078.492578] Consistencia: Hay una diferencia de: 190 ns.
[15185.952822] Modulo io_timer Descargado. Autor Gabriel Antonio Pichardo Raban
```

(Segundo estudiante: Mendelsson Nuñez)

1. Instalar cabeceras del kernel y herramientas de compilación:

```
sudo apt install linux-headers-$(uname -r) build-essential -y
```



```
mendel@Mubuntu: ~
mendel@Mubuntu:~$ sudo apt install linux-headers-$(uname -r) build-essential -y
[sudo] password for mendel:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
linux-headers-6.17.0-23-generic is already the newest version (6.17.0-23.23~24.04.1).
linux-headers-6.17.0-23-generic set to manually installed.
build-essential is already the newest version (12.10ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 43 not upgraded.
mendel@Mubuntu:~$
```

2. Crear Makefile con: obj-m := nombre_modulo.o:

```
mendel@Mubuntu: ~/practica_SO
mendel@Mubuntu:~$ mkdir ~/practica_SO
mendel@Mubuntu:~$ ls
Desktop Documents Downloads Music Pictures practica_SO Public snap Templates Videos
mendel@Mubuntu:~$ cd practica_SO/
mendel@Mubuntu:~/practica_SO$ nano io_timer_nunez.c
mendel@Mubuntu:~/practica_SO$
```

Primero creamos el archivo .c llamado io_timer_nunez.c

```
GNU nano 7.2 io_timer_nunez.c
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/ktime.h>
#include <linux/timex.h>

#define AUTHOR_NAME "Mendelsson Alejandro Nunez Montoya"

MODULE_LICENSE("GPL");
MODULE_AUTHOR(AUTHOR_NAME);
MODULE_DESCRIPTION("Modulo io_timer | Subsistema de Tiempo");

// Función de inicialización (Carga del módulo)
int init_module(void)
{
    u64 time_ns;
    u64 uptime_ns;
    u64 diff_ns;

    printk(KERN_INFO "--- Iniciando modulo io_timer ---\n");

    // Obtener el timestamp del kernel en nanosegundos
    time_ns = ktime_get_ns();

    // Obtener el tiempo desde el arranque (Uptime)
    // Nota técnica: Leer el archivo /proc/uptime directamente desde el kernel
    // es una mala práctica y riesgosa. La forma nativa y correcta en el kernel
    // para obtener el equivalente a /proc/uptime es usando ktime_get_boottime_ns().
    uptime_ns = ktime_get_boottime_ns();

    // Calcular si son consistentes
    if (uptime_ns > time_ns) {
        diff_ns = uptime_ns - time_ns;
    } else {
        diff_ns = time_ns - uptime_ns;
    }

    // Reportando ambos valores con printk
    printk(KERN_INFO "ktime_get() (Monotonico): %llu ns\n", time_ns);
    printk(KERN_INFO "Uptime (Boottime): %llu ns\n", uptime_ns);

    // Analisis de consistencia
    if (diff_ns > 0) {
        printk(KERN_INFO "Consistencia: Hay una diferencia de %llu ns.\n", diff_ns);
    } else {
        printk(KERN_INFO "Consistencia: Los valores son exactamente iguales.\n");
    }

    return 0;
}

// Función de limpieza (Descarga del módulo)
void cleanup_module(void)
{
    // Involucrando mensaje de descarga con el nombre del autor
    printk(KERN_INFO "Modulo io_timer descargado. Autor: %s\n", AUTHOR_NAME);
}

Help Write Out Where Is Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was Previous Next Back Forward Prev Word Next Word
```

Ingresamos al

archivo .c y escribimos todo el código que incluye las funciones init_module, cleanup_module y módulos de License, Author y Description.

```
mendel@Mubuntu:~/practica_SO$ nano Makefile
```

Creamos el archivo Makefile e ingresamos a él



```
mendel@Mubuntu: ~/practica_SO
GNU nano 7.2 Makefile *
obj-m := io_timer_nunez.o
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Con esto le indicamos al sistema de compilación del kernel cómo se llama nuestro archivo de código para convertirlo en un módulo

3. Compilar:

```
mendel@Mubuntu:~/practica_SO$ make -C /lib/modules/$(uname -r)/build M=$(pwd) modules
make: Entering directory '/usr/src/linux-headers-6.17.0-23-generic'
make[1]: Entering directory '/home/mendel/practica_SO'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M]  io_timer_nunez.o
MODPOST Module.symvers
CC [M]  io_timer_nunez.mod.o
CC [M]  .module-common.o
LD [M]  io_timer_nunez.ko
BTF [M] io_timer_nunez.ko
Skipping BTF generation for io_timer_nunez.ko due to unavailability of vmlinux
make[1]: Leaving directory '/home/mendel/practica_SO'
make: Leaving directory '/usr/src/linux-headers-6.17.0-23-generic'
```

4. Carga y verificación de mensajes printk:

```
mendel@Mubuntu:~/practica_SO$ sudo insmod io_timer_nunez.ko && sudo dmesg | tail -8
[ 8782.231022] --- Iniciando modulo io_timer ---
[ 8782.231027] ktime_get() (Monotonico): 8782249768442 ns
[ 8782.231030] Uptime (Boottime):      8782249768542 ns
[ 8782.231031] Consistencia: Hay una diferencia de 100 ns.
```

5. Verificación de la lista de módulos:

```
mendel@Mubuntu:~/practica_SO$ lsmod | grep io_timer_nunez
io_timer_nunez      12288  0
```

6. Descarga y verificación de mensaje cleanup:

```
mendel@mubuntu:~/practica_S0$ sudo rmmod io_timer_nunez && sudo dmesg | tail -5
[ 8782.231022] --- Iniciando modulo io_timer ---
[ 8782.231027] ktime_get() (Monotonic): 8782249768442 ns
[ 8782.231030] Uptime (Boottime):      8782249768542 ns
[ 8782.231031] Consistencia: Hay una diferencia de 100 ns.
[ 9007.200272] Modulo io_timer_descargado. Autor: Mendelsson Alejandro Nuñez Montoya
```

D. Emulación vs Paravirtualización en VirtualBox

9. Ejecutar `lspci -v` dentro de la VM e identificar qué dispositivos muestran vendor 'InnoTek'/'Oracle' (emulados) y cuáles son 'VirtIO' (paravirtualizados).

Ejecución de `lspci -v`:

```
mendel@mubuntu:~$ lspci -v
00:00.0 Host bridge: Intel Corporation 440FX - 82441FX PMC [Natoma] (rev 02)
Flags: fast devsel

00:01.0 ISA bridge: Intel Corporation 82371SB PIIX3 ISA [Natoma/Triton II]
Flags: bus master, medium devsel, latency 0

00:01.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01) (prog-if 8a [ISA Compatibility mode controller, supports both channels switched to PCI native mode, supports bus mastering])
Flags: bus master, fast devsel, latency 64
I/O ports at 01f0 [size=8]
I/O ports at 03f4
I/O ports at 0170 [size=8]
I/O ports at 0374
I/O ports at d000 [size=16]
Kernel driver in use: ata_piix
Kernel modules: pata_acpi

00:02.0 VGA compatible controller: VMware SVGA II Adapter (prog-if 00 [VGA controller])
Subsystem: VMware SVGA II Adapter
Flags: bus master, fast devsel, latency 64, IRQ 18
I/O ports at d010 [size=16]
Memory at e0000000 (32-bit, prefetchable) [size=128M]
Memory at f0000000 (32-bit, non-prefetchable) [size=2M]
Expansion ROM at 000c0000 [virtual] [disabled] [size=128K]
Kernel driver in use: vmwgfx
Kernel modules: vmwgfx

00:03.0 Ethernet controller: Red Hat, Inc. Virtio network device
Subsystem: Red Hat, Inc. Virtio network device
Flags: bus master, fast devsel, latency 64, IRQ 19
I/O ports at d020 [size=32]
Memory at f0200000 (32-bit, non-prefetchable) [size=8K]
Capabilities: <access denied>
Kernel driver in use: virtio-pci

00:04.0 System peripheral: InnoTek Systemberatung GmbH VirtualBox Guest Service
Flags: fast devsel, IRQ 20
I/O ports at d040 [size=32]
Memory at f0400000 (32-bit, non-prefetchable) [size=4M]
Memory at f0000000 (32-bit, prefetchable) [size=16K]
Kernel driver in use: vboxguest
Kernel modules: vboxguest

00:05.0 Multimedia audio controller: Intel Corporation 82801AA AC'97 Audio Controller (rev 01)
Subsystem: Dell 82801AA AC'97 Audio Controller
Flags: bus master, medium devsel, latency 64, IRQ 21
I/O ports at d100 [size=256]
I/O ports at d200 [size=64]
Kernel driver in use: snd_intel8x0
Kernel modules: snd_intel8x0

00:06.0 USB controller: Apple Inc. KeyLargo/Intrepid USB (prog-if 10 [OHCI])
Flags: bus master, fast devsel, latency 64, IRQ 22
Memory at f0804000 (32-bit, non-prefetchable) [size=4K]
Kernel driver in use: ohci-pci

00:07.0 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
Flags: medium devsel, IRQ 9
Kernel driver in use: piix4_smbus
Kernel modules: i2c_piix4

00:08.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)
Subsystem: Intel Corporation PRO/1000 MT Desktop Adapter
Flags: bus master, 66MHz, medium devsel, latency 64, IRQ 16
Memory at f0820000 (32-bit, non-prefetchable) [size=128K]
I/O ports at d240 [size=8]
Capabilities: <access denied>
Kernel driver in use: e1000
Kernel modules: e1000

00:0b.0 USB controller: Intel Corporation 82801FB/FBM/FR/FW/FRW (ICH6 Family) USB2 EHCI Controller (prog-if 20 [EHCI])
Flags: bus master, fast devsel, latency 64, IRQ 19
Memory at f0840000 (32-bit, non-prefetchable) [size=4K]
Kernel driver in use: ehci-pci

00:0d.0 SATA controller: Intel Corporation 82801HM/HEM (ICH8M/ICH8M-E) SATA Controller [AHCI mode] (rev 02) (prog-if 01 [AHCI 1.0])
Flags: bus master, fast devsel, latency 64, IRQ 21
I/O ports at d248 [size=8]
I/O ports at d250 [size=4]
I/O ports at d258 [size=8]
I/O ports at d260 [size=4]
I/O ports at d270 [size=16]
Memory at f0842000 (32-bit, non-prefetchable) [size=8K]
Capabilities: <access denied>
Kernel driver in use: ahci
Kernel modules: ahci
```


Al ejecutar `lspci -v` dentro de la VM, se listan todos los dispositivos PCI reconocidos por el kernel. En la salida se observa un dispositivo con vendor InnoTek Systemberatung GmbH en la dirección 00:04.0, correspondiente al "VirtualBox Guest Service", que es un dispositivo emulado típico de VirtualBox. No se encontraron dispositivos con vendor Oracle. Por otro lado, en 00:03.0 aparece un controlador de red con vendor Red Hat, Inc y la descripción "Virtio network device", lo que indica que se trata de un dispositivo paravirtualizado VirtIO. El resto de los dispositivos pertenecen a emulación genérica de hardware, no específica de VirtualBox.

10. Para cada dispositivo VirtIO identificado, ejecutar `lsmod | grep virtio` y `modinfo` del módulo correspondiente.

Comando `lsmod | grep virtio`:

```
mendel@mubuntu:~$ lsmod | grep virtio
mendel@mubuntu:~$
```

Comando para listar módulos built-in que contienen "virtio":

```
mendel@mubuntu:~$ cat /lib/modules/$(uname -r)/modules.builtin | grep virtio
kernel/drivers/virtio/virtio.ko
kernel/drivers/virtio/virtio_ring.ko
kernel/drivers/virtio/virtio_pci_modern_dev.ko
kernel/drivers/virtio/virtio_pci_legacy_dev.ko
kernel/drivers/virtio/virtio_mmio.ko
kernel/drivers/virtio/virtio_pci.ko
kernel/drivers/virtio/virtio_balloon.ko
kernel/drivers/char/virtio_console.ko
kernel/drivers/iommu/virtio-iommu.ko
kernel/drivers/block/virtio_blk.ko
kernel/drivers/scsi/virtio_scsi.ko
kernel/drivers/net/virtio_net.ko
```

Comando `modinfo virtio_pci` para obtener información del módulo:

```
mendel@mubuntu:~$ modinfo virtio_pci
name: virtio_pci
filename: (builtin)
version: 1
license: GPL
file: drivers/virtio/virtio_pci
description: virtio-pci
author: Anthony Liguori <aliguori@us.ibm.com>
parm: force_legacy:Force legacy mode for transitional virtio 1 devices (bool)
```

Al ejecutar `lsmod | grep virtio` no se obtuvo ninguna salida, lo que indica que los controladores VirtIO no están cargados como módulos independientes. Esto es normal en Ubuntu 24.04, donde estos drivers se compilan de forma estática dentro del kernel. Para confirmar su presencia, se ejecutó `cat /lib/modules/$(uname -r)/modules.builtin | grep virtio`, mostrando múltiples módulos como `virtio_pci`, `virtio_ring` y `virtio_net`. Además, `modinfo virtio_pci` arrojó la descripción completa del driver, incluyendo su autor y licencia GPL. Por lo tanto, aunque `lsmod` no los liste, el sistema tiene disponible el soporte VirtIO necesario para el dispositivo de red paravirtualizado.

11. Comparar en `dmesg` los mensajes de inicialización de un dispositivo emulado vs uno VirtIO. ¿Cuál genera más líneas? ¿Qué implica eso sobre el overhead de la emulación completa?


```

mendel@Mubuntu:~$ sudo dmesg | grep -i e1000
[ 1.276014] e1000: Intel(R) PRO/1000 Network Driver
[ 1.276018] e1000: Copyright (c) 1999-2006 Intel Corporation.
[ 1.760145] e1000 0000:00:08.0 eth0: (PCI:33MHz:32-bit) 08:00:27:2d:ad:88
[ 1.760160] e1000 0000:00:08.0 eth0: Intel(R) PRO/1000 Network Connection
[ 1.774030] e1000 0000:00:08.0 enp0s8: renamed from eth0
[ 11.265539] Modules linked in: i2c_piix4 drm_ttm_helper vboxguest(OE) polyval_clmulni ttm i2c_smbus ghash_clmulni_intel joydev aesni_intel input_leds serio_raw mac_hid binfmt_misc sch_fq_codel msr parport_pc ppdev lp parport efi_pstore nfnetlink_dmi_sysfs ip_tables x_tables autofs4 hid_generic usbhid hid vga16fb ahci video vgastate wmi psmouse e1000 libahci pata_acpi
[ 25.396294] e1000: enp0s8 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: RX
mendel@Mubuntu:~$ sudo dmesg | grep -i e1000 > e1000_dmesg.txt
mendel@Mubuntu:~$ sudo wc -l e1000_dmesg.txt
7 e1000_dmesg.txt

mendel@Mubuntu:~$ sudo dmesg | grep -i virtio_net
[ 1.250230] virtio_net virtio0 enp0s3: renamed from eth0
mendel@Mubuntu:~$ sudo dmesg | grep -i virtio_net > virtio_dmesg.txt
mendel@Mubuntu:~$ sudo wc -l virtio_dmesg.txt
1 virtio_dmesg.txt

```

¿Cuál genera más líneas en dmesg? Respuesta: El dispositivo emulado e1000 (Intel PRO/1000) generó 7 líneas en la salida de dmesg, mientras que el dispositivo paravirtualizado `virtio_net` generó solamente 1 línea. Por lo tanto, el controlador emulado produce significativamente más mensajes de inicialización en el registro del kernel que su contraparte VirtIO.

¿Qué implica eso sobre el overhead de la emulación completa? Respuesta: La mayor cantidad de líneas en `dmesg` para el dispositivo emulado refleja un proceso de inicialización más complejo y largo. Cada línea suele corresponder a una operación o interacción con el hypervisor (detección del hardware, asignación de recursos PCI, configuración de la dirección MAC, establecimiento del enlace, etc.). Esto implica que la emulación completa introduce un mayor overhead porque el hypervisor debe simular fielmente el comportamiento de un dispositivo físico real, lo que genera más “trampas” (VM Exits) y más instrucciones que ejecutar. En cambio, el driver VirtIO se comunica mediante un protocolo de alto nivel más directo, reduciendo el número de operaciones necesarias y, por ende, el costo de virtualización.

12. Explicar con evidencia propia del laboratorio: ¿por qué la paravirtualización requiere un driver especial en el SO huésped mientras que la emulación completa no?.

La Emulación Completa: No requiere un driver especial porque el hypervisor (VirtualBox) engaña al sistema operativo huésped haciéndole creer que está interactuando con un hardware físico real y tradicional. Como evidencia de nuestro laboratorio, al listar los dispositivos PCI, vimos que Ubuntu reconoció la tarjeta emulada como un "Intel Corporation 82540EM Gigabit Ethernet Controller" y cargó su driver genérico predeterminado (`e1000`). El sistema operativo nunca se enteró de que estaba en una máquina virtual; simplemente usó el mismo controlador que usaría si tuviera esa tarjeta física conectada a su placa base.

La Paravirtualización: Sí requiere un driver especial porque, en este modelo, el sistema operativo huésped *sabe* que está siendo virtualizado y colabora directamente con el hypervisor para mejorar el rendimiento. No hay un hardware falso que engañe al SO. Como evidencia en el laboratorio, tuvimos que configurar explícitamente el adaptador como "Paravirtualized Network (virtio-net)" en VirtualBox, y el sistema operativo tuvo que utilizar una familia de drivers especializados (`virtio_pci` y `virtio_net`) diseñados exclusivamente para comunicarse a través de la API del hypervisor, saltándose la costosa traducción de instrucciones del hardware simulado.

E1. Experimentos del Subsistema de E/S: E/S de Tiempo: Temporizadores, RTC e Interrupciones

5.1 — Ejecutar `cat /proc/timer_list | head -80` para ver las fuentes de reloj disponibles. Ejecutar `cat /sys/devices/system/clocksource/clocksource0/available_clocksource` y `current_clocksource`. Documentar qué clocksource está activa en la VM.

Se ejecutó `cat /proc/timer_list | head -80` para visualizar las fuentes de tiempo del kernel:

```
mendel@Mubuntu:~$ sudo cat /proc/timer_list | head -80
[sudo] password for mendel:
Timer List Version: v0.10
HRTIMER_MAX_CLOCK_BASES: 8
now at 537169958497 nsecs

cpu: 0
clock 0:
 .base:      0000000054c5ad82
 .index:     0
 .resolution: 1 nsecs
 .get_time:   ktime_get
 .offset:     0 nsecs

active timers:
#0: <00000000e528a642>, tick_nohz_handler, S:01
# expires at 537171000000-537171000000 nsecs [in 1041503 to 1041503 nsecs]
#1: <0000000066344f1b>, hrtimer_wakeup, S:01
# expires at 537267666383-537267766382 nsecs [in 97707886 to 97807885 nsecs]
#2: <000000008586ff68>, hrtimer_wakeup, S:01
# expires at 537268204931-537268304930 nsecs [in 98246434 to 98346433 nsecs]
#3: <000000007a6b3131>, hrtimer_wakeup, S:01
# expires at 537413189079-537413689078 nsecs [in 243230582 to 243730581 nsecs]
#4: <00000000d3fd3ba5>, timerfd_tmrproc, S:01
# expires at 537428832000-537428832000 nsecs [in 258873503 to 258873503 nsecs]
#5: <0000000090199d38>, hrtimer_wakeup, S:01
# expires at 537639809168-537640809166 nsecs [in 469850671 to 470850669 nsecs]
#6: <0000000057c9b31d>, hrtimer_wakeup, S:01
# expires at 537659502468-537660002467 nsecs [in 489543971 to 490043970 nsecs]
#7: <00000000c9353af1>, hrtimer_wakeup, S:01
# expires at 537660272920-537660768919 nsecs [in 490314423 to 490810422 nsecs]
#8: <000000003dd9cab3>, hrtimer_wakeup, S:01
# expires at 537941266810-537945262809 nsecs [in 771308313 to 775304312 nsecs]
#9: <00000000e5f043ba>, hrtimer_wakeup, S:01
# expires at 538000330046-538004325045 nsecs [in 830371549 to 834366548 nsecs]
#10: <0000000021f42ede>, dl_task_timer, S:01
# expires at 538109507361-538109507361 nsecs [in 939548864 to 939548864 nsecs]
#11: <00000000fc4c7fb2>, watchdog_timer_fn, S:01
# expires at 540367269157-540367269157 nsecs [in 3197310660 to 3197310660 nsecs]
#12: <00000000d2457ab3>, hrtimer_wakeup, S:01
# expires at 541866486908-541866536908 nsecs [in 4696528411 to 4696578411 nsecs]
#13: <0000000090ce0aae>, hrtimer_wakeup, S:01
# expires at 539864437021-542364437021 nsecs [in 2694478524 to 5194478524 nsecs]
#14: <00000000c50b6143>, hrtimer_wakeup, S:01
# expires at 543107842554-543117842550 nsecs [in 5937884057 to 5947884053 nsecs]
#15: <0000000019578973>, hrtimer_wakeup, S:01
# expires at 544864429962-544864429962 nsecs [in 7694471465 to 7694471465 nsecs]
```

```
#16: <00000000a0e6761b>, hrtimer_wakeup, S:01
# expires at 545941203355-545949980354 nsecs [in 8771244858 to 8780021857 nsecs]
#17: <00000000da6ea964>, hrtimer_wakeup, S:01
# expires at 550590083944-550637799943 nsecs [in 13420125447 to 13467841446 nsecs]
#18: <00000000ee95beb6>, hrtimer_wakeup, S:01
# expires at 553281269344-553297644343 nsecs [in 16111310847 to 16127685846 nsecs]
#19: <00000000b1207ceb>, hrtimer_wakeup, S:01
# expires at 585940723702-586000709701 nsecs [in 48770765205 to 48830751204 nsecs]
#20: <00000000fe835d4f>, hrtimer_wakeup, S:01
# expires at 586000753640-587000753640 nsecs [in 48830795143 to 49830795143 nsecs]
#21: <00000000a4564358>, hrtimer_wakeup, S:01
# expires at 619290642938-619390642938 nsecs [in 82120684441 to 82220684441 nsecs]
#22: <00000000582164e0>, timerfd_tmrproc, S:01
# expires at 621428832000-621428832000 nsecs [in 84258873503 to 84258873503 nsecs]
#23: <000000002915d308>, timerfd_tmrproc, S:01
# expires at 621428832000-621428832000 nsecs [in 84258873503 to 84258873503 nsecs]
#24: <0000000003d8859a>, hrtimer_wakeup, S:01
# expires at 3649941449888-3650041449888 nsecs [in 3112771491391 to 3112871491391 nsecs]
#25: <00000000a44f26b1>, timerfd_tmrproc, S:01
# expires at 86961428832000-86961428832000 nsecs [in 86424258873503 to 86424258873503 nsecs]
clock 1:
 .base:      000000007d4e9769
 .index:     1
 .resolution: 1 nsecs
 .get_time:   ktime_get_real
 .offset:     1780885877492203906 nsecs
active timers:
#0: <000000007786c88d>, timerfd_tmrproc, S:01
# expires at 9223372036854775807-9223372036854775807 nsecs [in 7442485622192613404 to 7442485622192613404 nsecs]
#1: <00000000171a0541>, timerfd_tmrproc, S:01
# expires at 9223372036854775807-9223372036854775807 nsecs [in 7442485622192613404 to 7442485622192613404 nsecs]
#2: <0000000025f973d6>, timerfd_tmrproc, S:01
# expires at 9223372036854775807-9223372036854775807 nsecs [in 7442485622192613404 to 7442485622192613404 nsecs]
clock 2:
 .base:      00000000863a84bb
 .index:     2
```

Luego, mediante `cat`

`/sys/devices/system/clocksource/clocksource0/available_clocksource` se listaron las fuentes de reloj disponibles en la VM :

```
mendel@Mubuntu:~$ cat /sys/devices/system/clocksource/clocksource0/available_clocksource
tsc kvm-clock acpi_pm
```

Ver cuál es el reloj activo mediante `cat`

`/sys/devices/system/clocksource/clocksource0/current_clocksource:`

```
mendel@Mubuntu:~$ sudo cat /sys/devices/system/clocksource/clocksource0/current_clocksource
tsc
```

El archivo `/sys/devices/system/clocksource/clocksource0/current_clocksource` muestra que la fuente de reloj activa en la VM es `tsc` (Time Stamp Counter). Esto significa que el kernel está utilizando el contador de ciclos del procesador como referencia temporal, en lugar de `kvm-clock` u otras fuentes. Aunque en entornos virtualizados el TSC puede presentar desviaciones, VirtualBox lo emplea por su baja latencia y alta resolución.

5.2 — Medir la frecuencia del timer interrupt: monitorear `watch -n1 cat /proc/interrupts | grep -i timer` y calcular cuántas interrupciones por segundo genera. Esto refleja el valor HZ del kernel compilado (típicamente 250 o 1000)

Para medir la frecuencia con la que el hardware virtualizado interrumpe al procesador, ejecuté el comando `watch -n1 "cat /proc/interrupts | grep -i timer"`. Para demostrar el incremento, tomé dos capturas de pantalla consecutivas con exactamente un segundo de diferencia:

```
mendel@Mubuntu:~$ watch -n1 "cat /proc/interrupts | grep -i timer"
Every 1.0s: cat /proc/interrupts | grep -i timer          Mubuntu: Mon Jun  8 03:28:10 2026

 0:          124          0 IO-APIC  2-edge      timer
LOC:       325572       426410 Local timer interrupts
```

```
Every 1.0s: cat /proc/interrupts | grep -i timer          Mubuntu: Mon Jun  8 03:28:11 2026
 0:         124          0 IO-APIC  2-edge      timer
LOC:       325760      426606 Local timer interrupts
```

Cálculo

Primera captura (03:28:10):

LOC: 325572 (CPU0) y 426410 (CPU1)

Segunda captura (03:28:11):

LOC: 325760 (CPU0) y 426606 (CPU1)

Diferencias en 1 segundo:

CPU0: $325760 - 325572 = 188$

CPU1: $426606 - 426410 = 196$

Total de interrupciones LOC en 1 segundo: $188 + 196 = 384$

Comparando dos capturas con 1 segundo de diferencia, se observó que el contador LOC (Local timer interrupts) aumentó en 188 interrupciones para la CPU0 y 196 para la CPU1, totalizando 384 interrupciones por segundo. Este valor se aproxima a las 500 esperadas para un sistema con $HZ=250$ y 2 CPUs, considerando las latencias propias de la virtualización. Por lo tanto, se concluye que el kernel de Ubuntu 24.04 en la VM está compilado con $HZ=250$.

5.3 — Escribir un programa C que mida la resolución del reloj con `clock_getres(CLOCK_MONOTONIC, &res)` y `clock_getres(CLOCK_REALTIME, &res)`. Compilar con gcc y ejecutar. Documentar la resolución en nanosegundos de cada reloj.

Creemos un programa en C llamado “clock_resolution.c” que mide la resolución del reloj:

```
GNU nano 7.2                                clock_resolution.c *
#include <stdio.h>
#include <time.h>

int main() {
    struct timespec res;

    // Medir resolución del reloj de tiempo real
    clock_getres(CLOCK_REALTIME, &res);
    printf("Resolución de CLOCK_REALTIME: %ld nanosegundos\n", res.tv_nsec);

    // Medir resolución del reloj monotónico
    clock_getres(CLOCK_MONOTONIC, &res);
    printf("Resolución de CLOCK_MONOTONIC: %ld nanosegundos\n", res.tv_nsec);

    return 0;
}
```

[^]G Help [^]O Write Out [^]W Where Is [^]K Cut [^]T Execute [^]C Location M-U Undo
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify [^]/ Go To Line M-E Redo

Luego compilamos con gcc el programa:

```
mendel@Mubuntu:~/Practica-S0---Equipo-7/experimentos$ gcc clock_resolution.c -o clock_resolution
mendel@Mubuntu:~/Practica-S0---Equipo-7/experimentos$
```

Ejecutamos el programa:

```
mendel@Mubuntu:~/Practica-S0---Equipo-7/experimentos$ ./clock_resolution
Resolución de CLOCK_REALTIME: 1 nanosegundos
Resolución de CLOCK_MONOTONIC: 1 nanosegundos
```

Se obtuvo que la resolución de ambos relojes es de 1 nanosegundo en teoría, aunque en la práctica el kernel redondea a la granularidad del temporizador de hardware. Sin embargo, en sistemas con kvm-clock y high-resolution timers habilitados, clock_getres efectivamente reporta 1 ns. Esto demuestra que el subsistema de tiempo del kernel es capaz de ofrecer resoluciones nanosegundas, aunque las interrupciones del timer ocurran a 250 Hz. La alta resolución se logra mediante el uso del Time Stamp Counter (TSC) o kvm-clock, que se leen directamente sin esperar una interrupción.

5.4 — Ejecutar `sudo hwclock --show` para leer el RTC. Comparar con `date`. Explorar `cat /proc/driver/rtc` para ver los atributos del reloj de tiempo real. ¿Cuál es la diferencia entre el RTC y el reloj del sistema (wall clock)?

Mostramos el Real Time Clock (RTC) con `sudo hwclock --show`:

```
mendel@Mubuntu:~$ sudo hwclock --show
2026-06-08 04:36:49.139316+00:00
```

Mostramos el reloj del sistema con `date`:

```
mendel@Mubuntu:~$ date
Mon Jun  8 04:37:05 AM UTC 2026
```

Revisamos los atributos del RTC mediante `cat /proc/driver/rtc`:

```
mendel@Mubuntu:~$ cat /proc/driver/rtc
rtc_time      : 04:37:17
rtc_date      : 2026-06-08
alarm_time    : 00:00:00
alarm_date    : 2026-06-09
alarm_IRQ     : no
alarm_pending : no
update IRQ enabled : no
periodic IRQ enabled : no
periodic IRQ frequency : 1024
max user IRQ frequency : 64
24hr         : yes
periodic_IRQ  : no
update_IRQ    : no
HPET_emulated : no
BCD          : yes
DST_enable    : no
periodic_freq : 1024
batt_status   : okay
```

¿Cuál es la diferencia entre el RTC y el reloj del sistema (wall clock)?: La principal diferencia fundamental entre ambos es que el RTC es un dispositivo de hardware alimentado por una pequeña batería en la placa base. Por esta razón, el RTC sigue funcionando incluso cuando la computadora (o la máquina virtual) está completamente apagada, manteniendo la hora. En contraste, el reloj del sistema es un contador puramente software gestionado por el kernel. Este reloj se inicializa al arrancar el sistema, tomando la hora almacenada en el RTC, y es el encargado de proporcionar la hora a todos los procesos y aplicaciones mientras el equipo está encendido.

5.5 — Instalar ntpdate (sudo apt install ntpdate) y ejecutar sudo ntpdate -q pool.ntp.org. Registrar el offset entre el reloj de la VM y el servidor NTP. Las VMs frecuentemente tienen mayor drift que sistemas físicos.

Instalamos ntpdate:

```
mendel@Mubuntu:~$ sudo apt update && sudo apt install ntpdate -y
[sudo] password for mendel:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
13 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ntpsec-ntpdate ntpsec-ntpdig python3-ntp
The following NEW packages will be installed:
  ntpdate ntpsec-ntpdate ntpsec-ntpdig python3-ntp
0 upgraded, 4 newly installed, 0 to remove and 13 not upgraded.
Need to get 156 kB of archives.
After this operation, 607 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 python3-ntp amd64 1.2.2+dfsg1-4build2 [91.2 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 ntpsec-ntpdig amd64 1.2.2+dfsg1-4build2 [25.2 kB]
amd64 1.2.2+dfsg1-4build2 [25.2 kB]
```

Consultamos al servidor NTP:

```
mendel@Mubuntu:~$ sudo ntpdate -q pool.ntp.org
2026-06-08 05:14:49.885646 (+0000) +0.045096 +/- 0.089202 pool.ntp.org 50.43.156.177 s1 no-leap
```

La salida del comando muestra un offset de +0.045096 segundos (aproximadamente 45 milisegundos), con un margen de error de ± 0.089202 segundos. El valor positivo indica que el reloj del sistema está adelantado con respecto a la hora real de referencia. Este desvío (drift) es notablemente mayor al que se esperaría en un sistema físico, lo cual es típico en entornos virtualizados. La causa principal es que el hypervisor (VirtualBox) no permite al SO huésped acceder de forma directa y continua a los temporizadores de alta precisión del procesador (como el TSC), introduciendo latencias variables y pérdida de interrupciones del timer.

E2. Experimento de Virtualización: Paravirtualización del Reloj: TSC y kvm-clock

6.1 — Ejecutar dmesg | grep -iE 'tsc|hpet|pit|clocksource|kvm' dentro de la VM. Documentar qué fuente de tiempo detectó el kernel al arrancar. ¿Es diferente a lo que usaría un sistema físico bare-metal?


```

username@ubuntu:/practica_S0$ sudo dmesg | grep -iE 'tsc|hpet|pit|clocksource|kvm'
[sudo] password for username:
[ 0.000000] [Firmware Bug]: TSC doesn't count with P0 frequency!
[ 0.000000] Hypervisor detected: KVM
[ 0.000000] kvm-clock: Using msrs 4b564d01 and 4b564d00
[ 0.000005] kvm-clock: using sched offset of 4304171382 cycles
[ 0.000008] clocksource: kvm-clock: mask: 0xffffffffffffffff max_cycles: 0x1cd42e4dffb, max_idle_ns: 881590591483 ns
[ 0.000013] tsc: Detected 3593.246 Mhz processor
[ 0.036741] Booting paravirtualized kernel on KVM
[ 0.036744] clocksource: refined-jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 1910969940391419 ns
[ 0.037457] kvm-guest: PV spinlocks enabled
[ 0.108236] clocksource: tsc-early: mask: 0xffffffffffffffff max_cycles: 0x33cb66efe9d, max_idle_ns: 440795351205 ns
[ 0.220969] clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 1911260446275000 ns
[ 0.277323] clocksource: Switched to clocksource kvm-clock
[ 0.291038] clocksource: acpi_pm: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 2085701024 ns
[ 0.314083] clocksource: tsc: mask: 0xffffffffffffffff max_cycles: 0x33cb66efe9d, max_idle_ns: 440795351205 ns
[ 0.314124] clocksource: Switched to clocksource tsc
[ 5.507896] vmwgfx 0000:00:02.0: [drm] Capabilities: rect copy, cursor, cursor bypass, cursor bypass 2, alpha cursor,
extended fifo, pitchlock, irq mask, gmr, traces, gmr2, screen object 2, command buffers, command buffers 2, gobject,
[ 5.914513] kvm_amd: Nested Virtualization enabled
[ 5.914520] kvm_amd: Nested Paging disabled
[ 5.914536] kvm_amd: PMU virtualization is disabled
[ 35.344224] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 1227884903 wd_nsec: 1227884288
[ 76.382404] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 4174426890 wd_nsec: 4174424956
[ 243.960990] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 49977556749 wd_nsec: 49977533345
[ 4261.933419] vboxdrv: TSC mode is Invariant, tentative frequency 3593185477 Hz
username@ubuntu:/practica_S0$

```

R= Nuestro Kernel al arrancar la primera fuente de tiempo que detectó fue la kvm-clock, pero a medida que el sistema terminaba de arrancar fue cambiando de fuente de tiempo, pasando por jiffies, clocksource kvm y para finalizar en la fuente de tiempo TSC, ¿Esta es diferente a la que usaría un sistema físico Bare-metal? Pues en el arranque si sería diferente ya que nuestra maquina virtual utiliza de buenas a primeras su fuente de tiempo kvm-clock pero mientras nuestro sistema se va levantando el sistema switchea la fuente de tiempo para finalizar en una fuente de tiempo TSC, la cual la utilizan los sistemas bare-metal modernos, así que podríamos decir que cuando encendemos nuestra maquina en los primeros instantes es diferente la fuente de tiempo pero una vez inicializada la maquina utilizan la misma fuente de tiempo, TSC.

6.2 — Suspender la VM con Machine > Save State durante exactamente 2 minutos (medir con reloj externo). Al reanudar, ejecutar inmediatamente date y hwclock --show. ¿El reloj del sistema detectó el tiempo transcurrido? ¿Hay diferencia entre RTC y reloj del sistema?

```

username@ubuntu:/practica_S0$ date
Sat Jun  6 10:59:12 PM UTC 2026
username@ubuntu:/practica_S0$ sudo hwclock --show --directisa
2026-06-06 22:53:22.171798+00:00
username@ubuntu:/practica_S0$ date
Sat Jun  6 10:59:26 PM UTC 2026
username@ubuntu:/practica_S0$ sudo hwclock --show --directisa
2026-06-06 22:53:27.105449+00:00
username@ubuntu:/practica_S0$

```

(Cuadro verde: Tiempo registrado al inicio) (Cuadro Azul: Tiempo registrado luego de pausar la VM y esperar 2min)

¿El reloj del sistema detectó el tiempo transcurrido?

R= No para el reloj del sistema operativo no ocurrió nada

¿Hay diferencia entre RTC y reloj del sistema?

R= El reloj de nuestro sistema depende de los ciclos activos de la VM, mientras que el reloj RTC depende de una pila en nuestra placa madre, en nuestro caso la VM simula el hardware por hypervisor, en nuestro caso al pausarlo nuestro entorno virtual se congela completamente para mantener una consistencia y por ello ambos relojes no detectan el tiempo transcurrido.

6.3 — Experimento de precisión: registrar date +%s%N (nanosegundos), ejecutar sleep 5, registrar de nuevo. Calcular cuántos nanosegundos pasaron vs los 5000000000 esperados.

```
username@ubuntu:/practica_S0$ START=$(date +%s%N); sleep 5; END=$(date +%s%N); ELAPSED=$((END - START)); DIFF=$((ELAPSED - 5000000000)); echo -e "Inicio: $START\nFinal: $END\nPasaron: $ELAPSED ns\nDif $DIFF ns "
```

Inicio: 1780788081956262823
Final: 1780788092334615424
Pasaron: 10378352601 ns
Dif 5378352601 ns

(ejecutamos el comando de esta manera para que sea lo más preciso posible)

Comparar ejecutando el mismo experimento en el anfitrión. ¿La VM es menos precisa?

```
PS C:\Users\Carolina> $START = [DateTimeOffset]::Now.ToUnixTimeMilliseconds(); Start-Sleep -Seconds 5; $END = [DateTimeOffset]::Now.ToUnixTimeMilliseconds(); $ELAPSED = $END - $START; $DIFF = $ELAPSED - 5000; Write-Host "Inicio: $START ms\nFinal: $END ms\nPasaron: $ELAPSED ms\nDif: $DIFF ms"
```

Inicio: 1780788662010 ms
Final: 1780788667018 ms
Pasaron: 5008 ms
Dif: 8 ms

```
Inicio: 1780788662010 ms  
Final: 1780788667018 ms  
Pasaron: 5008 ms  
Dif: 8 ms
```

R= En **Bare Metal**, la precisión depende puramente de la granularidad del reloj del sistema operativo, En cambio, en la **Máquina Virtual**, la desviación suele ser un poco más impredecible porque el tiempo de la VM compite con el resto de los programas que tenemos abiertos en windows. Si nuestra PC física está muy ocupada, la VM tardará más nanosegundos en enterarse de que el sleep terminó. La máquina física demostró ser infinitamente más precisa, con una desviación casi imperceptible de 8 ms. La Máquina Virtual sufrió una degradación severa de precisión, tardando 10.3 segundos en completar la tarea, evidenciando la sobrecarga de planificación que introduce el hipervisor cuando compite por los recursos del hardware real.

Pregunta Integradora

El kernel usa el TSC (Time Stamp Counter) del CPU para medir el tiempo con alta precisión. Cuando una VM corre en VirtualBox, el acceso a la instrucción RDTSC puede ser interceptado por el hypervisor.

¿Por qué esto puede causar que el tiempo en una VM corra de forma inconsistente?

R= La instrucción RDTSC es una instrucción de ensamblador que permite al kernel leer directamente el número de ciclos de reloj que han pasado desde que se encendió el procesador físico. Es extremadamente rápida porque accede a un registro del hardware. Cuando VirtualBox intercepta esta instrucción, ocurre lo siguiente:

La penalización del Hypervisor: Cada vez que el kernel de Linux ejecuta RDTSC, el hipervisor detiene la ejecución de la VM, toma el control, calcula el tiempo virtualizado que le corresponde mostrar a la VM y le devuelve el resultado. Este desvío por software tarda cientos de ciclos de CPU más que una lectura directa de hardware. El tiempo que tarda el propio hipervisor en calcular el tiempo altera la medición.

¿Qué es kvm-clock y cómo resuelve este problema mediante paravirtualización?

R= kvm-clock es una fuente de tiempo de tipo paravirtualizada. A diferencia de TSC o HPET, kvm-clock es un driver diseñado específicamente para que el kernel de Linux de la VM y el hipervisor (KVM/VirtualBox) hablen el mismo idioma nativamente.

La paravirtualización significa que el sistema operativo invitado sabe que está virtualizado y coopera con el anfitrión para ser más eficiente. kvm-clock resuelve las inconsistencias del tiempo eliminando por completo la necesidad de interceptar la instrucción

```
username@ubuntu:/practica_S0$ cat /sys/devices/system/clocksource/clocksource0/available_clocksource
```

tsc kvm-clock acpi_pm

Este comando demuestra que el kernel de Linux tiene cargado en el núcleo el driver paravirtualizado kvm-clock, listo para ser usado si el TSC falla o si el hipervisor se lo pide.

¿Por qué después de suspender y reanudar una VM el reloj del sistema puede mostrar la hora correcta pero los intervalos medidos con CLOCK_MONOTONIC pueden estar desincronizados?

R= Cuando suspendes y reanudas una Máquina Virtual, los dos relojes principales del kernel reaccionan de formas completamente opuestas debido a su diseño:

El Reloj del Sistema (CLOCK_REALTIME)

Es el reloj que mide el tiempo real del mundo. Por qué muestra la hora correcta al reanudar: Al despertar de la suspensión, el hipervisor le inyecta inmediatamente la hora actual del anfitrión al kernel de la VM, o el sistema de sincronización hace un salto brusco en el tiempo para ponerse al día. Su prioridad es reflejar la realidad del usuario.

2. El Reloj Monotónico (CLOCK_MONOTONIC)

Es un reloj diseñado estrictamente para medir intervalos de tiempo (como un cronómetro). Su única regla inquebrantable es que nunca puede ir hacia atrás y nunca puede dar saltos bruscos. Siempre avanza de forma lineal hacia adelante basándose en los tics del procesador desde que el sistema arrancó. Por qué se desincroniza cuando la VM se suspende, los tics del procesador virtual se detienen por completo. Como CLOCK_MONOTONIC no permite saltos artificiales para "corregir" el tiempo, el cronómetro simplemente se congela.

Al reanudar, CLOCK_REALTIME da un salto mágico hacia adelante para ponerse al día con el mundo real, pero CLOCK_MONOTONIC continúa contando justo desde donde se quedó. La diferencia entre ambos se agranda exactamente por la cantidad de tiempo que la VM estuvo suspendida.

```
--- Reloj del sistema RealTime ---  
Mon Jun  8 11:45:27 PM UTC 2026  
--- Tiempo de ejecucion del kernel ---  
26608.60 77592.34
```

```
--- Reloj del sistema RealTime ---  
Mon Jun  8 11:49:11 PM UTC 2026  
--- Tiempo de ejecucion del kernel ---  
26678.88 77801.93
```

Para comprobar lo anteriormente mencionado ejecutamos los comandos `date` y `cat /proc/uptime` para ver el tiempo real del sistema y el tiempo de ejecución del kernel respectivamente, y hemos suspendido la VM, en la imagen con bordes verde vemos el antes y en la azul el después, vemos que la hora real del sistema ha hecho un salto mientras que la ejecución del kernel se ha mantenido prácticamente igual. Esto es claramente diferente a nuestro Experimento E2 en el cual ambas horas se mantuvieron iguales, debido a que en el experimento congelamos la maquina y por lo tanto los ciclos del procesador lo cual evitaba que nuestro tiempo de ejecución en el kernel avanzase, pero en este caso lo que hemos hecho fue suspender la máquina virtual para que nuestra VM siguiera simulando los ciclos del procesador.